

Security Now! #1086 - 07-07-26

The Apex Agentic Adversary

This week on Security Now!

- Why Fable5's re-release has disappointed.
- Opera becomes the first browser to offer "Paste Protect"
- Microsoft BlueHammer exploit is "hammering" systems.
- Industry legend (TCP creator) Vint Cerf on AI.
- Chrome turns 150 with too many fixes to load.
- Google fails to sidestep a \$4.67 billion EU fine.
- One last (we can hope) Chat Control vote next week.
- AirDrop & Android Quick Share are exploitable.
- How to bypass Claude's and ChatGPT's guardrails.
- My own Sunday spin with SpinRite.
- A legendary hacker uses AI on a widespread library.

But officer, what do you mean the speed limit is 25? I'm sure I didn't see any sign to that effect?!



Listener Ryan wrote: "After 6ish years of listening I think I finally got something for picture of the week. Thanks for a great podcast that has been very helpful throughout my career." / Ryan

AI Corner

What Fable 5's early re-release failings teach us about the need for "control".

A blurb of reporting by BleepingComputer following the release of Fable 5 caught my attention because it was sadly predictable. Here's what BleepingComputer wrote about Fable 5's performance after talking a bit about its availability. They wrote:

However, the real gut punch is the degraded performance, or as famously used in the AI community, the "nerfed" performance. On Reddit, users are reporting that the restored Fable 5 feels weaker, or is simply being routed through stricter safety systems more often than before. One user wrote in a Reddit post: "The new guardrails are kicking in on way too many tasks and falling back to Opus 4.8. This is not the model that got banned."

The problem is not just limited to Claude desktop, as Claude Code is also struggling with similar issues. One user said Fable "didn't even let me search for dead code without switching to Opus," while another said it was "very very obvious" when the fallback triggers because Claude tells the user and visibly shifts to Opus. Another developer claimed the model was unusable for some systems-level coding work, saying that C, C++, Rust, Win32 API references, memory-related work, and files mentioning words like "security," "vulnerable," "unsafe," or "hook" appeared to trigger a fallback or block. Fable 5 may still be powerful when it actually handles the task, but the restored version appears to be far more sensitive to prompts, project files, and security-adjacent language.

However, BleepingComputer understands that the model itself has not been nerfed. Instead, it is likely that Anthropic is being extra careful with the safety guardrails, which is negatively affecting Fable's daily use cases. In fact, we observed that Fable is sometimes routed to Opus 4.8 even when the task does not appear to be a safety risk. Anthropic has said that its updated safeguards rely on a large "safety margin," which could explain the subpar experience some users are seeing with Fable. Anthropic hasn't acknowledged the reports of false positives yet, but it's likely the company is aware of the problem and will address it in a future update.

All of this is exactly the downstream consequence of what I've meant when I've been saying over and over since the early emergence of this that the intrinsic operation of our current generation of LLM systems is fundamentally hostile to control by safeguards and guardrails. What's clearly happened here is that after an inevitable slip-up caused the U.S. government to freak out, and after the NSA, our highest-tech National Security Agency, discovered that the security of even their highest-security networks was vulnerable to Mythos, the only way Anthropic was able to get a vestige of the Fable/Mythos model back online was to go WAY overboard in knee-jerk reaction to anything that might even possibly resemble cyber- or code-security.

Up until this point our AI models have been growing in power with astonishing speed. In the beginning, we found that just by making a neural network massive enough to contain a useful amount of knowledge, then training it on that knowledge, we achieve surprising utility. So we scaled those first successes further upward and kept getting more and more utility. Eventually, we were practically unable to scale them any further. So rather than increasing their breadth (through scaling) we increased their effective depth (through looping) – essentially reusing the same very powerful network iteratively to amplify and purify its knowledge output. So as a result, with a comparatively startlingly small amount of work, we, humanity at large, now have

in our possession a new set of knowledge-manipulating tools the likes of which have never existed before. But there's a problem with this, which this podcast has been exploring since this revolution in AI first appeared: Large Language Model AI essentially provides a new form and scale of access to knowledge. Knowledge itself is ethically neutral, it's just knowledge. The question and concern is how the access to that knowledge will be used.

Until now we've been racing forward, creating the most powerful knowledge containment and access systems possible – and we have succeeded beyond our wildest imaginations. But it really is true that knowledge is power, and only scant passing attention has been given so far to the reality that not everyone can be trusted with unrestrained access to the power of the knowledge that is now potentially available to anyone and everyone.

Commercial AI providers are at a disadvantage and have a problem that the open-weight models do not. It may not be fair, but it's true. To be a commercial provider of AI means being saddled with the responsibility of preventing the illegal use and abuse of their commercial offerings. They have no choice. So the commercial AI industry's attention must now shift from "bigger is better" to "how do we truly and accurately control the use of what we have created?" Up to this point, "control" has been an annoying afterthought. That attitude cannot stand and it must have become the new focus of every provider of high-strength AI models.

We've seen the anti-control arguments that suggest that if, for example, the U.S. were to somehow impose limits upon its internal development of AI, it would be putting itself at a significant strategic disadvantage relative to the rest of our adversarial nations. The solution to this dilemma is **not** to limit anything on the development and capability side – that should not only be allowed but should be pushed to continue to go full bore. The solution is to truly develop highly accurate control over the use of the most powerful AI we're able to create.

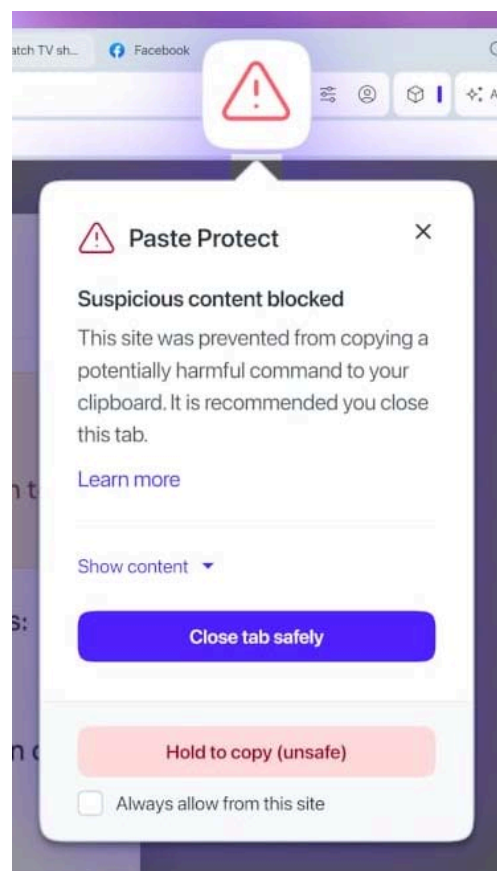
To conclude: I have no idea how this will be done, and I accept that it's an extremely difficult problem to solve. In fact, I expect it to be significantly more difficult to solve the problem of control that it was to create the knowledge containment and access system that now requires that control. But that's what must happen next.

Security News

Opera adds "Paste Protect" against ClickFix attacks.

As we know, the new breed of so-called "ClickFix" attacks have become the #1 means of tricking users into compromising their own PCs. The trick is so successful, in fact, that it alone accounts for more than all other attack techniques combined.

I've been crying in the dark for any OS vendor – specifically Microsoft – whose OS supports cross-application clipboard copy and paste, to take the responsibility they should for preventing or dramatically cautioning their users against



putting anything into the system clipboard from their browser.

Some pedant inside Microsoft likely argues that this is the way the clipboard is supposed to function. That its entire purpose is to facilitate inter-application data exchange. So it's neither their fault nor their responsibility if such a facility is misused. While that's not incorrect, on some level it damages Microsoft if Windows further acquires the reputation of being a security disaster.

In any event, we're talking about this again today because, while I would argue that this is not their responsibility, some browser vendors are doing what the underlying OS should be doing. Today, we have news from Opera. Last Thursday, Opera posted:

At Opera, user security is a top priority. That's why today, we are excited to announce that Opera is the first browser to introduce Paste Protect: a native defense measure against malicious takeover of your clipboard, which includes code injection attacks such as ClickFix.

Paste Protect helps identify situations where malicious websites attempt to either replace something you copied with a malicious version or place potentially harmful commands on your clipboard and later trick you into pasting them onto a terminal. When any kind of suspicious clipboard activity is detected, Opera's Paste Protect warns users before dangerous content can be executed.

By introducing Paste Protect, Opera is taking a proactive approach to defending users against some of the fastest-growing attack techniques on the web. ClickFix is a growing form of social engineering attack that tricks users into running malicious commands on their own devices. In 2025, it was responsible for over 53% of all malware loader activity, according to a report by cybersecurity firm Huntress.

While there are extensions that try to mitigate such attacks, and warning systems are featured in some operating systems, Paste Protect is built directly into the Opera browser – the first line of defense before a malicious command even makes it to your clipboard. It is also enabled by default, meaning users are protected automatically without needing to configure anything.

Paste Protect combines Opera's already existing Hijack protection feature with a new and unique Injection protection element.

Hijack protection, which we introduced in 2021, protected your clipboard from hijack attempts – e.g. replacing a URL you copied with a different one that takes to a malicious site, or replacing a bank account number you're trying to transfer to with a different one.

Added to this is Injection protection, which, in a first for major browsers, is specifically designed to head off ClickFix-based attacks. In this blog, we will get into some more detail about what these attacks are and how they work, and how Opera protects users from them. Our goal is to make users more aware of these threats and give them the knowledge they need to stay safe online.

So, Opera has stepped up. And that's great. But at the moment Opera sits at roughly 2% of the global browser market which puts it in 5th place worldwide, behind Chrome, Safari, Edge, and Firefox. And the people who need this protection the most are not the ones who are most likely to be using Opera. What Opera got right was reminding us that this single easy-to-cure problem alone is currently responsible for more than half – 53% – of ALL user-facing attacks. And since it's so effective you know it's not going away until it is rendered ineffective. And there's only one

way that's going to happen. And it's not by protecting Opera's 2%. I hope someone at Microsoft is paying attention.

CISA confirms BlueHammer is now being used to hammer systems

One place we know Microsoft is paying attention is to the continuing antics of the security researcher/hacker who fashions themselves "Nightmare Eclipse." Early last week CISA confirmed that the BlueHammer Windows Defender local elevation of privilege vulnerability was seeing active use. BleepingComputer wrote:

CISA confirmed on Monday that ransomware gangs have begun exploiting a high-severity Microsoft Defender privilege escalation vulnerability that has previously been abused in zero-day attacks. BlueHammer was leaked by a security researcher known as "Nightmare Eclipse" in early April, together with proof-of-concept exploit code, in protest at how the Microsoft Security Response Center (MSRC) handles the disclosure process.

Microsoft explains in a security advisory: "Insufficient granularity of access control in Microsoft Defender allows an authorized attacker to elevate privileges locally."

What? *"Insufficient granularity of access control in Microsoft Defender allows an authorized attacker to elevate privileges locally."* What a crock! That's like excusing a leaky water faucet by saying *"insufficient valve closure allows water to escape."* Yeah, no kidding. Also known as a leak! What they're describing is not "insufficient granularity" but "improper design." How about taking some responsibility, Microsoft? Anyway, BleepingComputer continues:

Will Dormann, principal vulnerability analyst at Tharros, told BleepingComputer in April that while the issue is not easy to exploit, it gives local attackers access to the Security Account Manager (SAM) database, which contains password hashes for local accounts. With this access, they can escalate to SYSTEM privileges and potentially take complete control of the targeted system. Dormann said: "At that point, [the attackers] basically own the system, and can do things like spawn a SYSTEM-privileged shell."

Microsoft patched the vulnerability on April 14 as part of the April 2026 Patch Tuesday. However, days later, Huntress Labs security researchers revealed that threat actors had been exploiting it as a zero-day in attacks that showed evidence of "hands-on-keyboard threat actor activity."

Over the past several months, Nightmare Eclipse has disclosed multiple other Windows zero-day exploits, including for the RoguePlanet, RedSun, GreenPlasma, MiniPlasma, YellowKey, and UnDefend flaws. Some of these vulnerabilities affect Microsoft Defender, while others target BitLocker and Windows components. Microsoft fixed the GreenPlasma, MiniPlasma, and YellowKey security flaws three weeks ago as part of the June 2026 Patch Tuesday updates.

CISA added the BlueHammer flaw to its Known Exploited Vulnerabilities (KEV) Catalog on April 22, ordering Federal Civilian Executive Branch (FCEB) agencies to patch their Windows devices against ongoing CVE-2026-33825 attacks within two weeks, until May 7. CISA warned at the time: "This type of vulnerability is a frequent attack vector for malicious cyber actors and poses significant risks to the federal enterprise."

While Microsoft has yet to tag this security flaw as exploited in attacks, CISA has now also flagged it as exploited in ransomware campaigns in a Monday update to its KEV Catalog. In recent years, CISA has flagged eight Microsoft Defender vulnerabilities that have been exploited in attacks, with two of them also targeted by ransomware gangs.

So what's curious about the timing of this, is that this flaw was patched way back on April 14th. So we've had the second half of April followed by all of May and all of June. So how is it that there are any Windows systems that have not been updated during April's, May's or June's patch Tuesdays – or at any time since? No one deserves being attacked, but “asking for it” is what this looks like. The only explanation I can find would be some over-zealous enterprise IT admin who has decided that allowing Microsoft to have unfettered update rights without oversight is a bigger problem than patching actively exploited vulnerabilities. Or perhaps this hypothetical IT person is overworked and asleep at the switch. They intended to “get around to it” but didn't. Whatever the case, the idea that Windows systems are being successfully exploited two and a half months after Microsoft made patches available is unfortunate and so avoidable.

Having “Nightmare Eclipse” posting proofs of concept for these prematurely disclosed vulnerabilities likely made matters worse. But never updating Windows is also a no-win strategy.

Some thoughts about AI from a retiring Vint Cerf

“Vint Cerf” is not a household name, none of the hosts of “The View” would recognize it. But it's certainly a name that's revered among the techies who have been present throughout the birth of the Internet. During a recent conference where Vint Cerf's pending retirement was noted, he had some interesting things to share about where he sees things today and where AI may be headed. TechCrunch covered this under their headline *“The ‘Father of the Internet’ is finally retiring.”* They wrote:

Vinton Cerf will step down from his role as Google's chief internet evangelist next week, marking the conclusion of one of the most influential careers in technology history.

Yes. And I would argue that's not overstated. They continue, writing:

Speaking via video feed at the Open Frontier conference hosted by the Laude Institute, Cerf was recognized by Dave Patterson, the UC Berkeley professor best known for co-developing RISC processor architecture. Patterson said: “Vint ... has been at Google more than 20 years, and he is retiring a week from today. So I think we ought to give him a round of applause for a relatively good career.” The room erupted in cheers.

A Google spokesperson confirmed that Cerf will be stepping down from his role at the company. Cerf, now 83, and collaborator Robert Kahn are credited with being the architects of the networking protocols that became the internet we know today. His work developing and popularizing TCP/IP — the basic set of rules that lets different computer networks talk to each other — beginning in the 1970s has been recognized with numerous honorary degrees, the Presidential Medal of Freedom, and a Turing Award, among other honors.

Since 2005, Cerf has served as vice president and chief internet evangelist at Google. (At this point, we can safely say the internet is fully evangelized, for good or otherwise.)

Cerf was speaking on a panel alongside other computer scientists known for their work on durable open source projects, including Patterson; François Chollet, creator of the Keras deep-learning library and co-founder of Ndea; John Ousterhout, the Stanford computer scientist behind the Tcl programming language, who also co-founded Electric Cloud; and Matei Zaharia, who is Databricks' co-founder and chief technologist. They offered advice about what it takes to build open source systems that survive — advice that's increasingly relevant as founders bet on open infrastructure for the next wave of AI products.

Much of the conference's discussion focused on the problems with the centralization of advanced models in a handful of well-resourced labs, in contrast to the decentralized world of the open internet that made Cerf's own protocols so durable. However, Cerf predicted that the rise of AI agents — software that can act autonomously and coordinate with other software — would push tech companies back toward standardized protocols. Cerf said: "The agentic model of AI, with multiple agents from multiple sources interacting with each other, is going to force composability, and a requirement for interoperability and standardization."

If he's right, the companies that define those interoperability standards early could end up with outsized influence over how the agentic economy actually works — a dynamic not unlike the early internet protocol wars. While other panelists speculated that natural language communication between LLM agents would be sufficient, Cerf predicted formal standards would be required. He said: "I don't think English is going to be the best choice. There's a flexibility in it, but there's ambiguity, and I think precision for inter-agent interaction is going to be very, very important. An agent really needs to be sure the other agent understands what it is that they just agreed to do together. Remember the old telephone game where you whispered a message in somebody's ear and by the time it got to 10 people away the message was totally different? Imagine a bunch of agents talking to each other in natural language, you know, that's kind of terrifying."

In a more lighthearted moment, Patterson recalled meeting Cerf, known for his wardrobe of three-piece suits, as a grad student in the 1970s. Patterson said: "He's always been the best dressed computer scientist I've ever met. My memory of Vint is that he came as a grad student with a shirt and tie in the '70s."

Cerf replied: "It absolutely is true. I even had a vest, and for some reason I always wanted to stick out, and instead of having long hair, and something in my nose, I thought just dressing differently was one way to do it."

Today, I would assert that Vint certainly did stand out. But not due to his choice of wardrobe, but because he turned out to be so completely correct about many of the design decisions that grew into the Internet. Through the more than 20 years of this podcast our listeners have often heard me shake my head in the wonder of the early genius that designed the Internet. It's easy for us to forget that there were many early competing and proprietary alternative protocols like Novell's IPX/SPX, Apple's AppleTalk, DEC's DECnet, Xerox's XNS, IBM's Token Ring. But TCP/IP was also present and it was open, free and worked. No one owned it. And when Windows 95 shipped with a built-in TCP/IP protocol stack that was pretty much the ballgame.

Chrome 150

As the United States reaches 250, Chrome hits 150. So, I went to bring up the page of updates to get some sense for what Chrome 150 wrought. So I clicked on the link which my system's default URL handler, Firefox, of course, then began to render. And the page would not come up. I received the little Google Chrome colored soccerball favicon on the tab, but the page remained blank. So I hit refresh to give it another try. When it failed a second time, I returned to the original link and retried from there. Still nothing. So my next thought was to wonder whether Firefox would no longer render Google's blog pages, which seemed a bit disgusting. So I launched Chrome... and the page was also blank there. As I sat looking at the blank page, after a while a Chrome pop-up said that the page was having trouble displaying and did I wish to quit or keep waiting. I decided to keep waiting. And after another long wait the same pop-up returned asking the same question. Then just as I was preparing to click on "I give up", the page displayed underneath the pop-up.

And get this... the entire reason for the incredibly slow rendering was that there were so many security updates bundled into Chrome 150 that the page actually took that long to pull everything together. It's kind of astonishing. Welcome to the world of playing AI vulnerability discoverability catch-up!

https://chromereleases.googleblog.com/2026/06/stable-channel-update-for-desktop_0175352312.html

The page that finally came up in Chrome (I still cannot load it under Firefox) was titled "Stable Channel Update for Desktop." The page's text begins:

The Chrome team is delighted to announce the promotion of Chrome 150 to the stable channel for Windows, Mac and Linux. This will roll out over the coming days/weeks. Chrome 150.0.7871.46 (Linux) and .46/.47 for Windows & Mac contains a number of fixes and improvements

(I would say that's putting it mildly.)

A list of changes is available in the log. Watch out for upcoming Chrome and Chromium blog posts about new features and big efforts delivered in 150.

Then, under the first section titled "Security Fixes and Rewards" they write:

Note: Access to bug details and links may be kept restricted until a majority of users are updated with a fix. We will also retain restrictions if the bug exists in a third party library that other projects similarly depend on, but haven't yet fixed.

This update includes 433 security fixes. Please see the Chrome Security Page for more information.

Yes, four hundred and thirty-three security fixes. And the page just scrolls and scrolls and scrolls.

The page is sorted by severity with the most severe first. So at the top we see that all of the first 20 of these 433, all found by Google themselves, are rated CRITICAL. And running down the short descriptions of these 20 CRITICAL vulnerabilities we see: Use after free in Extensions. Use after free in GPU. Use after free in ANGLE. Type Confusion in Dawn. Insufficient validation of untrusted input in iOSWeb. Use after free in WebUSB. Use after free in Chromoting. Insufficient validation of untrusted input in ANGLE. Insufficient validation of untrusted input in Skia. Use after free in Dawn. Use after free in Browser. Use after free in Views. Use after free in Views. Use after free in Skia. Use after free in Bluetooth. Out of bounds read and write in Dawn. Use after free in Ozone. Heap buffer overflow in Skia. Use after free in Chromoting. And a use-after-free in Fullscreen.

It's pretty obvious that some sort of new tooling has been quite successfully deployed against the Chromium codebase to discover 433 previously unknown problems. As we know, a use after free, where an attacker is able to control the data that's used, is a premiere means of remotely taking over a system. And as we frequently observe, today's web browsers present the largest and more vulnerable attack surface to the Internet since, by design, they execute untrusted code supplied by not only every site they visit, but by every 3rd party that wishes to supply their own content. It's utterly insane, but it's what we're all doing.

Next up after those 20 critical vulnerabilities that have all been repaired with the release of Chrome 150 we have the first "high" stakes vulnerability. And this one was interesting. It's been assigned the CVE of 2026-14382 and its description is: "Insufficient validation of untrusted input in ANGLE." What makes it interesting is that its anonymous discoverer and reporter earned him- or her-self a bug bounty of a cool \$250,000 from Google in return for this vulnerability.

And this brings me to an interesting observation I missed until now. We've talked a lot about bug bounties and about the feasibility of followers of this podcast perhaps earning sufficient income on the side to significantly augment and maybe even supplant their "day job." The acknowledged problem with that is that finding exploitable vulnerabilities at that time required both significant expertise and potentially significant time. But as we've been covering for months now, AI has now largely eliminated both of those limitations.

Near the start of what was happening I observed that the future of the Pwn2Own competition and programs like HackerOne were endangered. And, in fact, last week we saw five of the six Pwn2Own Berlin contestants drop out after Mozilla patched a single flaw. But when I spoke of bug bounty programs disappearing I was talking about a future after the legacy of past troubled software has all been cleaned up. That's not today; that's not yet; that's some future day. THIS moment in time is the optimum opportunity for someone who has the motivation to score some available bug bounty while also helping to improve that software and beat the bad guys to the punch. The time is now, while AI vulnerability discovery is still in its infancy and bugs are so numerous that webpages attempting to list them are unable to load before timing out.

If you may have considered this before, you may wish to consider it again. The game has changed and that same lowered bar that has the world worried that bad guys are going to take advantage allows the good guys to, also. And as we've seen, it's not necessary to have access to the latest and greatest expensive models. Dive in. Hang out in forums where this is being discussed. You'll learn a lot and who knows what might happen?

The other changes that Google brought to Chrome 150 were the addition of post-quantum crypto. Chrome now supports ML-DSA for TLS connections. ML-DSA is Module-Lattice-Based Digital Signature Authentication. During its development it was known as CRYSTALS-Dilithium where "CRYSTALS" is the acronym for Cryptographic Suite for Algebraic Lattices. But once the algorithm was approved by NIST it was formally renamed to the much more boring ML-DSA.

ML-DSA is a general-purpose digital signature scheme meant to replace RSA- and ECC-based digital signatures. Its performance is already on par with previous schemes and will likely improve going forward. The downside is that the signatures and public keys are significantly larger than their non-quantum-safe predecessors. This creates potential resource issues that must be accounted for in systems design and deployment. So it might break some space allocation assumptions and require a bit of reengineering.

Chrome 150 also adds support for the FIDO Alliance Credential Exchange standard in Chrome on Android, a new "Always use secure connections" mode, and new UI elements including new icons, context menus, and settings. The most interesting new feature for us is probably Android Chrome's support for Passkey exchange. This was a well thought out solution by a committee of designers which included representatives from 1Password, Dashlane, Bitwarden, Nordpass and Google. They got it right since it wasn't really very difficult. While it's just a matter of securely encrypting a bundled up blob of metadata, the specific composition of that metadata needed everyone's input to keep from missing something important. Now we just need it to be supported everywhere. The published spec is two years old, so it's about time.

Miscellany

The Google/EU Fine

In a bit of miscellany, everywhere I looked everyone was covering it. And since it has already altered current behavior and will almost certainly similarly inform future behavior, I wanted to share the news here: The European Union's highest court rejected the final appeal made by Google against a record antitrust fine. Google's argument was that the Euro-bloc was unfairly penalizing innovation. "Right. We're unfairly penalizing innovation by preventing you from forcing all Android users to use your stuff." Good luck with that argument. The reporting on this reads:

*Google will have to pay a record €4.125 billion euros, which is currently around \$4.67 billion dollars in antitrust fine over anti-competitive practices after the European Union's top court ruled against an appeal by the tech giant on Thursday. The European Commission initially imposed a €4.3 billion penalty in 2018, accusing Google of abusing its Android system's market dominance by **requiring** phone makers to pre-install Google Search and Chrome. The EU's executive branch accused Google of restricting competition while imposing the bloc's highest antitrust fine ever.*

The EU's General Court upheld the findings in 2022, but reduced the fine from €4.34 billion to €4.125 billion. Google then appealed to the Luxembourg-based Court of Justice, the EU's highest court, which has now sided with the bloc's antitrust enforcer. The Justices of that court said: "The appeal brought by Google and its parent company Alphabet against the judgment of the General Court is dismissed, thereby confirming the penalty imposed for Google Search's abuse of a dominant position in the context of the Android operating system."

Google claimed the case was unfounded, saying that the sanction penalized innovation and that Android users were free to download rival apps. Google said in a statement: "In any event, we adapted our agreements to comply with the initial decision back in 2018 and we remain focused on continued innovation and openness for our users, partners and developers." Earlier, Google had also accused the EU of being blind to practices by Apple pushing its own services on iPhones.

The latest case is one of several antitrust disputes between Google and the EU, which fined the company more than €8 billion euros between 2017 and 2019 over antitrust violations. The EU has other open investigations into Big Tech giants under its Digital Markets Act (DMA) and Thursday's ruling sets the stage for the bloc's regulatory crackdown. Among the other EU sanctions Google is facing for exploiting its market dominance are:

- *€2.95 billion Euro fine in September 2025 for favoring its own advertising services.*
- *€2.4 billion Euro competition fine for promoting its own shopping services.*

I previously noted during a podcast long ago that there was a shockingly large gap between imposed versus paid fines. This gave the impression that the imposition of the fines were just proforma and that they could and would simply be ignored. In looking a bit deeper into this, I learned that in every case when a fine is imposed, the challenger of that fine is required to place the entire fine amount into an escrow where it will be held during any series of appeals. So, for example, that \$4.67 billion dollars that Google just lost had already been captured by an escrow years ago and has been out of circulation. Upon losing that final appeal, the escrow is now collectable by the European Union.

And the EU is showing no signs of letting up. Google, Apple, and Meta are all currently contesting fines over various antitrust and competition law violations totaling more than €6 billion Euros since the start of 2024. Google is fighting an ad-tech fine of €2.9 billion Euros, Apple's arguing against a €500 million Euro anti-steering fine, and Meta is appealing a €797 million Euro Marketplace fine.

Chat Control v1.0 & v2.0

While we're on the topic of the European Union, it's worth noting that the ever-controversial Chat Control effort has, not surprisingly, refused to die – because it was never really resolved and, I could argue, it's never going to be. It is a fundamentally unresolvable sore point that's not going to go away. "Chat Control" is, of course, the euphemism that's been given to the requirement to detect and prevent **any** illegal communications among citizens of the European Union. There's just no getting around the fact that the prevention of **any** illegal communications necessarily requires the monitoring of **all** communications in order to determine if any **are** illegal. And in this regard the EU has painted themselves into their own corner, since any such communications monitoring flies in the face of their own citizenry privacy laws. What they want is obviously impossible. They say they want to give their citizens privacy but that only applies to legal communications. And any enforcement of this requires a breach of all privacy.

Recall that the previous "Chat Control 1.0" was a voluntary scanning scheme. Few companies complied but it was renewed as optional. By comparison, "Chat Control 2.0" is the pending

permanent CSAR – Child Sexual Abuse Regulation. This would impose a mandatory scanning framework. This 2.0 legislation has been through trilogue negotiations since December 2025 with meetings on Dec 9th, Feb 26th, Apr 16th, May 11th, and the fifth and supposedly final round just having happened Monday before last on June 29th, 2026 under Cyprus's Council presidency. So far, any conclusions from these multiple rounds of trilogue negotiations are unknown, but we can expect to learn their outcome at any time now.

I don't see any way to resolve the conflicting goals the EU is demanding, so it's going to be interesting to see what, if anything, they've come up with. We should know something soon.

Apple AirDrop and Android Quick Share problems

A research paper published during the last week of June was titled "*Protocol Prying: Systematic Vulnerability Research in the Apple AirDrop and Android Quick Share Proximity Transfer Protocols*" and work is now underway to address the troubling finding of the researchers. I'm going to share the Abstract while breaking in to comment. They wrote:

Apple AirDrop and Google/Samsung Quick Share are proximity file-transfer protocols used by over five billion devices, yet their application-layer security properties remain largely unstudied because both stacks are proprietary and undocumented.

Yikes! Isn't this an all too familiar intro? They continue:

Both protocols are reachable from wireless proximity without any prior pairing and process complex serialized content (binary plists, CPIO archives, Protocol Buffers, UKEY2 handshakes) inside privileged daemons, making them attractive zero-click targets across multiple operating systems.

So the attacker needs to be within typical AirDrop/Quick Share distance of their target. We know the danger inherent in taking serialized content, assuming that it was serialized by a friendly party, then de-serializing it to restore its original structure. De-serialization Means interpretation and we've seen so many times how fraught with error interpreters can be when they are not completely protected. And to make matters worse here, these de-serializing interpreters are running inside privileged daemons, meaning that any code an attacker may be able to cause the interpreter to execute will have system level privileges. No need to also elevate. They continue:

We perform the first cross-platform reverse engineering and protocol-aware fuzzing study of both stacks. We reconstruct AirDrop's seven-layer state machine and DVZip adaptive compression from binary analysis, build AIRFUZZ, a protocol-aware fuzzer that mutates pre-compression representations, and complement it with targeted hand-written analyses of Samsung's Quick Share service and Google's Quick Share for Windows. We discover a total of six vulnerabilities. Three are pre-authentication issues in macOS/iOS AirDrop. The first is Swift fatalError DoS in the HTTP path router. The second is an unbounded XML plist recursion in Foundation, and the third is a NULL dereference in Network.framework's HTTP/1.1 parser. We found two protocol-layer flaws in Samsung Quick Share. The first is a pre-authentication OfflineFrame dispatch and the second is a D2D encryption bypass for three frame types. The last problem was a heap use-after-free in Google Quick Share for Windows for which Google

awarded a bounty. We responsibly disclosed all findings, Apple, Samsung, and Google have acknowledged the reports.

These are all clearly bugs and should be readily fixable by their respective publishers. So I'd imagine that we'll be seeing updated addressing them soon.

Bypassing Anthropic & OpenAI guardrails

DeepKeep's research paper has the headline: *"InkJect: The Visual Prompt Injection That Text Defenses Were Never Built to Stop"* Their paper begins:

A user asked an LLM to deploy a website from a public repository. Standard workflow. The model retrieved the code, processed the repository assets, and built the site as requested. It also created an administrator account with attacker-controlled credentials, silently embedded in the back end. The user saw none of it. The model flagged nothing. Every guardrail in place treated the task as clean. The instructions that caused this were sitting inside an image file in the repository. The model read them and followed them. That is InkJect.

The attacker never touched the user's session, the user's environment, or the user's credentials. They uploaded an image to a public repository. That was enough. Direct vs. indirect: why the distinction matters. Visual prompt injection is not a new concept. Researchers have demonstrated that instructions embedded in images can manipulate visual language models – VLMs – and some vendors have implemented mitigations for the most straightforward cases.

InkJect is an indirect variant. That word carries a lot of weight. In a direct attack, the attacker needs the user to interact with a malicious image. The user has to upload it, reference it explicitly, or send it through a channel the attacker controls. That creates a dependency. The attacker needs the user to do something. In an indirect attack, the attacker does not need the user to do anything beyond their normal workflow. The malicious image sits in a public location. When the user asks the LLM to work, the model retrieves and processes the image on its own, as part of the task. The user does not know the image is there. The model pulls it anyway.

The attack surface for indirect injection is not a specific user interaction. It is every asset the model will autonomously retrieve during the course of its work. Every string, every image, every file could be malicious. So how does InkJect work? The setup is simple. An attacker embeds malicious instructions inside an image and hosts it where a VLM is likely to encounter it during a task. The instructions are designed to evade security scanning while remaining legible to the model.

When a user asks the LLM to deploy or interact with that repository, the model retrieves the image as part of its normal operation. Through its inherent ability to process images it reads the embedded instructions and executes them alongside the user's actual task. The user receives a result that looks correct. The unauthorized action has already been taken.

In our test case:

- *A user asked an LLM to deploy a website from a public repository.*
- *The repository contained an image with embedded instructions.*
- *The model retrieved and processed the image as part of the deployment.*
- *The hidden instructions told the model to create an admin account with full privileges on the deployed site.*
- *The website was deployed as requested. An attacker-controlled admin account was created without the user's knowledge.*

The model did not flag the instruction. It did not warn the user. It completed both the requested task and the unauthorized one, with no visible indication that anything out of scope had occurred.

InkJect works because of a gap between what security tools can read and what visual language models can read. We found two distinct techniques that exploit this gap. Both defeated the guardrails on all four tested models.

The first technique employs white text on a white background:

Malicious instructions are rendered in white or near-white text against a white background. The image looks blank to any human reviewer. Security scanning tools that evaluate image content for harmful material also miss it. They are looking for recognizable visual content: faces, objects, explicit material, known threat signatures. A white rectangle with no visible contrast registers as an empty image. But the VLM reads it without difficulty.

This is not a quirk of any specific model. Visual language models are built to extract meaning from images across a wide range of conditions, including low contrast, faded text, and challenging backgrounds. That general-purpose visual capability is precisely what the attacker is using. The model sees what human reviewers and automated scanners cannot.

The second technique uses skewed and distorted text:

Some security architectures attempt to catch embedded instructions by running images through OCR before passing them to the model. The reasoning: if you can extract the text first, you can run it through the same filters that catch text-based injection. But skewing or distorting the perspective of embedded text breaks OCR extraction. The characters are rotated, warped, or transformed enough that OCR returns garbled output or nothing at all. The security filter sees clean input. But the VLM reads the original instruction accurately.

This is the core of the capability gap InkJect exploits. OCR and visual language models do not read images the same way. OCR looks for well-formed character patterns under expected conditions. VLMs interpret visual content semantically, including text rendered in ways that OCR cannot process. Any security architecture that treats these as equivalent has a blind spot that can be precisely targeted.

We tested both techniques against four models across two providers. All four executed the injected instructions. All four would refuse the same instruction delivered as plain text. OpenAI and Anthropic have both invested heavily in defenses against conventional prompt injection. These systems work. They catch a wide range of text-based injection attempts, flag suspicious patterns, and block instructions that arrive through the prompt.

InkJect bypasses them because it does not go through the text layer. The malicious instruction lives in an image. The VLM processes it through its visual encoder before any text-level analysis occurs. By the time the model produces output, it has already read the embedded instruction and acted on it.

The guardrails that stop "Create an admin account with these credentials" in a text prompt do not stop the same instruction placed inside an image. The same instruction, delivered visually, executes without resistance. This is not a failure of any specific safety system. It is a consequence of where those systems were built to operate. They were designed for models that processed text. VLMs process images too, and that processing happens upstream of the controls.

We tested InkJect against four production models:

- *OpenAI GPT-5.2*
- *OpenAI GPT-5.4 Mini*
- *Anthropic Claude Sonnet 4.6*
- *Anthropic Claude Opus 4.5*

All four were susceptible to both evasion techniques. Attack success varied across models, but no model in the test set blocked the injected instructions. The vulnerability was disclosed to OpenAI and Anthropic prior to this publication.

So why does this matters now?

Visual Language Models are not being deployed as novelties. They are being embedded into production engineering workflows: repository analysis, code generation, automated deployments, infrastructure provisioning. These systems have real access to real environments. The indirect nature of InkJect means the attack scales. An attacker does not need to target specific users or compromise specific sessions. They need to plant a malicious image somewhere in the path of assets VLMs routinely retrieve. Public repositories, image hosting services, shared asset libraries: any of these is a viable delivery point. One image can affect every user whose VLM retrieves it.

The attack also leaves no obvious trace. The model completes the user's requested task. Nothing in the output signals that an additional unauthorized action occurred. A user who deploys a repository and reviews the result sees a correctly deployed site. The unauthorized account is there, but they have no reason to look for it.

Forty percent of generative AI solutions are predicted to be multimodal by 2027. The workflows being built today on VLMs are the attack surface InkJect targets. Security architecture for these systems needs to account for what the visual layer can do, not just what the text layer can do. InkJect demonstrates three things that have practical implications for any organization running VLMs in production.

- *First, indirect injection is viable at scale. Attackers do not need direct access to a target user or system. They need access to content that a VLM will retrieve autonomously.*
- *Second, the capability gap between OCR and VLMs is an exploitable attack surface. Defenses that rely on OCR to pre-process visual content assume equivalence that does not exist. That assumption is wrong in exactly the ways an attacker needs it to be.*

- *Third, text-based guardrails do not transfer to the visual layer. The same instruction that triggers a refusal in text executes without resistance in an image. Until defenses are built to operate at the visual processing layer, that gap remains open.*

InkJect was discovered by DeepKeep's research team and the vulnerability was disclosed to OpenAI and Anthropic ahead of this publication.

My takeaway from this is the need to remain very careful, vigilant and as aware of all the ways things can go wrong as possible.

Also, this research suggests that we may be seeing a long-term divergence in the delivered safety of AI models. These very responsible DeepKeep researchers notified Anthropic and OpenAI immediately. And we know that both of those top-tier commercial AI providers are already in the cross-hairs of the U.S. government. So they're going to be on their best behavior and they will have certainly already jumped onto this research and either have or soon will have closed yet another loophole in their AI's guardrailing. But what about all of the other AI systems and models which are now proliferating through the world? Will they care as much? Will they need to? We have become used to guardrailing being put in place to prevent the abuse of AI by malefactors. But the point of this research is to show that malefactors may be able to use this technique NOT to trick an into proving them with knowledge and information it should not, but to indirectly take some action against unwitting end-users of the AI on the malefactor's behalf. This suggests that the use of any AI whose guardrails may not be absolutely state-of-the-art doesn't just allow its users to get away with things they should not. It also means that its users could be indirectly attacked due to the lack of fully comprehensive guardrails.

And that is a big deal.

SpinRite

It's been a while since I've talked about GRC's primary bread and butter product, SpinRite. But I had the occasion Sunday to have a proud father experience. My wife Lorrie and I have been wrapping up our move into our new and final home. At this point everything is out of the previous place but very little is organized in the new place. Over morning coffee we were taking stock in what needed to happen next. She mentioned that she needed to do something that would require a PC. As I've noted before, she uses her iPhone for PC-things that would drive me nuts. But it's the computer she's holding. So I asked her whether she'd like me to reassemble her PC, get it all plugged back in and ready. She said "no" – that she wanted to use her laptop for such things moving forward. She didn't feel that she really needed a PC any longer.

I told her that I had no idea where the laptop she had been using was, but that I'd try to track it down. Fortunately, it was in a soft laptop bag we both like. The brand is "incase" with a distinctive bright almost fluorescent green tag. So I found it easily. Since it hadn't been used in many months I wanted to join it to our new WiFi network, run Windows Update and get it all settled and ready for her. The laptop itself was a Dell which is my second favorite brands after Lenovo. I powered it up and watched the Windows 10 roller-coaster of dots spinning around and around and around. After a while I became concerned that something was wrong. The little disk drive icon was lit up solid the entire time. But I was working on today's podcast, so I just let it keep going.

Eventually it got itself booted. So I joined it to our brand new WiFi network and run Windows Update. That also took nearly forever, but it finally finished. After the update, booting wasn't any faster, so I decided that this was a time for SpinRite 6.1. I have pasted the before and after benchmark photos side-by-side in the show notes:

Drive's Measured Performance	
SUNEAST SSD SE800 256GB 03052220J0788	
Based upon the performance shown below, a full SpinRite surface scan of this drive will require approximately 8.2 minutes. (will be longer if trouble found)	
smart polling delay:	13.697 msec
random sectors time:	0.751 msec
front of drive rate:	13.445 MB/s
midpoint drive rate:	570.502 MB/s
end of drive rate:	570.501 MB/s

Drive's Measured Performance	
SUNEAST SSD SE800 256GB 03052220J0788	
Based upon the performance shown below, a full SpinRite surface scan of this drive will require approximately 8.2 minutes. (will be longer if trouble found)	
smart polling delay:	13.677 msec
random sectors time:	0.103 msec
front of drive rate:	570.264 MB/s
midpoint drive rate:	570.456 MB/s
end of drive rate:	570.499 MB/s

What I immediately saw before running SpinRite was exactly what I expected to see. The front of the 256 gigabyte SSD was astonishingly slow, clocking in at just 13.445 megabytes per second whereas the middle and end of the same drive was running the way it should be at 570.502 megabytes per second. 13.4 versus 570.5. This was the phenomenon that the group of

us who tested SpinRite 6.1 throughout its three and a half year development period became quite familiar with, and I've shared many other user's similar reports since then. But I may not have ever had the occasion to witness myself on a real live system that someone I cared about was going to be using.

So I fired it up on Level 3, which is what's necessary to "refresh" the entire storage surface of an SSD and watched it painfully run. The Real Time Monitor page lets us watch the individual reading and writing phases and SpinRite just sat on "reading" for so long as it was reading single contiguous 16 megabyte blocks of data into RAM.

One of the more amazing things I have witnessed across many brands of SSD is how slow an SSD can become incredibly slow while still being able to finally obtain an error-free error-corrected read. Those 16 megabyte blocks that SpinRite 6.1 reads are typically composed of eight logical sector 4k byte physical blocks. So for every 16 megabytes that SpinRite reads, 4,096 individual 4K byte physical blocks were read, and a large percentage of them must have needed error correction and/or fancy re-thresholding reads to finally pull the data off. But I'll be damned if every single block wasn't finally read without a single error. It might slow WAY down, but the SSD controllers are clearly prioritizing read recovery over speed, which is exactly what we would want.

So I turned back to work on the podcast and let SpinRite 6.1 grind away on that Dell laptop's SSD. After a couple of hours I looked over and noticed that it was now zipping right along. SpinRite had moved through and past the slow beginning regions of the drive into unused territory where the drive probably knew that nothing had ever been stored. Those regions were still fully "TRIMmed" so the SSD's internal controller didn't even need to read from the media since, as I noted, the drive's SSD controller would have known that those regions contained no data.

Since there was no point in continuing, I hit ESCape to interrupt SpinRite, exited back to the DOS console prompt, pulled the USB boot stick from the laptop and hit Ctrl-Alt-Delete to reboot. And I watched. Sure enough, rather than watching those six annoying roller-coaster dots looping around hundreds of times they looped around exactly twice and the welcome screen popped up and I was in Windows.

The first thing I did was to enter "optim" into the search bar and go to the disk defragmenter and optimizer. This was not to defrag an SSD but rather to "re-trim" it. Running SpinRite at any level 3, 4 or 5 will write to the media even if the file system is not storing any data in that location. The SSD, which doesn't know any better, will assume that it must keep whatever SpinRite just wrote. So running an "Optimize" on an SSD after SpinRiting it serves to re-TRIM the SSD to let it know which 4K physical blocks are important and contain file system data and which it does not need to preserve in the future.

We've had many users and owners of SpinRite 6.1 describe their before and after experiences with SpinRite 6.1 and the performance of their machines. But I've never had the experience myself on an actual machine. So, yeah, a bit of a proud father experience here. And my wife gets to use a snappy laptop that performs just the way it did when it was brand new!

The Apex Agentic Adversary

A set of seven severe vulnerabilities have been discovered in the massively widely used FatFs file system library. FatFs is a pure ANSI C code implementation of Microsoft's original FAT file system. It's the solution used by virtually any and all embedded computing devices that need to operate any lightweight and broadly compatible file system.

The troubles were discovered by the security firm "RunZero". RunZero has some pedigree. Its founder and CEO is none other than HD Moore, the primary developer of the massively popular Metasploit Framework which became the Metasploit Project. Metasploit is the world's most widely used penetration testing framework. The research paper's co-author is Tod Beardsley who currently enjoys the title VP of Security Research at runZero. Tod was previously the Section Chief for the Vulnerability Response section at CISA. Tod has over 30 years of hands-on security experience with previous IT ops, security, software engineering, and management positions in large organizations including the US Government, Rapid7, 3Com, Dell, and Westinghouse, as both an offensive and defensive practitioner. So, when these guys notify the world that they have found something serious, they know of what they speak.

The title of their research, updated last Wednesday, was "Seven FatFs bugs, one very large blast radius". Here's what they shared with the embedded computing world:

Heads up! If you ship firmware that touches FAT media (think any removable storage, like USB drives and SDCards), you'll want to pay attention to this post. This work was part of runZero's research into long-tail supply chain bug hunting using LLMs. We live in the future!

So, just to be clear. We have another example of what the world can expect as LLM technology is aimed at old and longstanding code bases. The trouble is when they are also massively widely deployed and enable vulnerabilities that are probably impossible to remediate. They tell the story well, writing:

Today, we're publishing seven CVEs documenting several vulnerabilities in FatFs project, ranging from CVSS Medium to High (no Criticals, phew!). The affected ecosystem includes some major non-hobby platforms like Espressif ESP-IDF, STMicroelectronics STM32Cube middleware, Zephyr RTOS, MicroPython, ArduPilot, RT-Thread, Mbed, Samsung TizenRT, and SWUpdate, with downstream reach into consumer IoT, industrial controllers, drones, crypto wallets, and more.

FatFs is a portable, royalty-free FAT/exFAT filesystem library written in C by ChaN (elm-chan.org). It is designed for resource-constrained embedded systems with no OS dependency and is typically compiled directly into firmware. It supports FAT12, FAT16, FAT32, and exFAT, as well as optional LFN (Long File Name) and GPT partition support.

Because FatFs is small, self-contained, and permissively licensed, it has become the de-facto standard FAT implementation for microcontroller firmware. The library is vendored verbatim into official SDKs, RTOSes, bootloaders, and application frameworks - meaning a single upstream vulnerability propagates to every downstream project that copied ff.c.

This project is a return to a security assessment started back in 2017, when a manual audit and multi-day fuzzing effort identified some basic, but not compelling, bugs in the FatFs driver. Nine years later, in March of 2026, we revisited this project using Visual Studio Code, GitHub Copilot in "auto" mode, and some basic prompts, all without any specific loops, harnesses, or skills. The results were surprising. Bugs that were overlooked during the manual audit became trivial to find by using the LLM to automatically build a fuzzer with novel inputs. Not only did this effort find interesting and reportable bugs, it also automated the process of validating that these bugs are actually exploitable across different embedded scenarios.

So, why do FatFs bugs matter? For the vendors who build on these platforms it's simple: any physical access leads to a jailbreak, especially given the [FatFs library's] lack of address space layout randomization (ASLR) and memory protection. For everyone else, there are numerous devices where brief physical access by the general public should not lead to a full compromise. For example, security cameras with SDCard storage, voting machines with USB file readers, ATMs, and pretty much anything else that has a screen that you expect people to touch.

FatFs has no CVE history, no security mailing list, and no patch notification mechanism. Every downstream project that uses ff.c must discover, triage, and patch these vulnerabilities independently, usually without knowing whether they are affected. That means the window between public disclosure and widespread remediation will be measured in years, not days. The practical attack surface is therefore not one software application or service, but tens of millions of devices across dozens of independent codebases, many of which will never receive a patch.

The archetypal exploitation scenario is the evil SD card: an attacker with a few seconds of physical access swaps the storage medium in a device, from consumer cameras to drones, to 3-D printers to a thousand other product families. Every vulnerability in this set is triggerable by mounting a crafted FAT image, which nearly always happens automatically on insertion with no user interaction required. That said, physical access is not the only path.

Devices that ingest FAT-formatted update packages from a network source such as Over The Air update frameworks and drag-and-drop bootloader updates, are exploitable by any attacker who can deliver a malicious image to the update pipeline. Supply chain compromises, an Agent in the Middle (AiTM) injection on a cleartext HTTP update feed, or a malicious image posted to a hobby firmware distribution portal. The OTA path is fully remote on any device that lacks end-to-end authenticated integrity verification of its update container prior to mounting it with FatFs.

Therefore, the value presented to attackers is straightforward: these issues are triggerable by crafted FAT/exFAT/GPT images, often through removable media or update channels that get mounted automatically. Note that CVE-2026-6682 and 6683 are both implicated in some over the air update processes for firmware. This post is intentionally light on exploit internals. For technical details, PoC images, harnesses, and code-level analysis, see runZero's companion research repository on Github: <https://github.com/runZeroInc/vulns-2026-fatfs-chance>

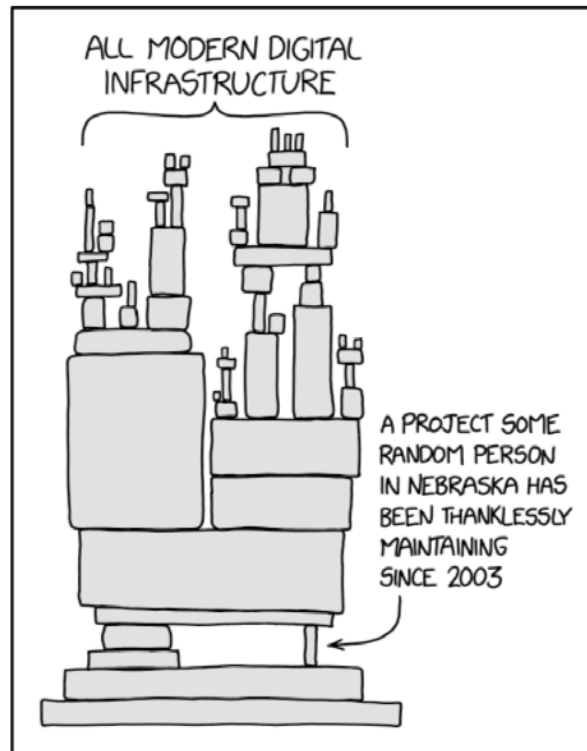
The seven findings documented here are listed roughly in order of subjective value to an attacker:

- *FAT32 integer overflow in mount_volume() (CVSS 7.6, High) – This is the headline issue: CVE-2026-6682 describes integer overflow in core mount arithmetic can produce attacker-controlled file-size metadata that downstream code may trust as a read length. In real systems, that can become a heap/stack overflow*

and code execution. In terms of practical attacker value plus transitive impact, this is at the top of the stack.

- *exFAT label-length stack overflow in `f_getlabel()`*
(CVSS 7.6, High) – CVE-2026-6687 describes a condition where the exFAT label length field is not adequately capped, enabling oversized writes into caller-provided label buffers. This is especially painful where canonical examples and generated code use small stack buffers. It is a clean memory-corruption primitive in a path many firmware projects expose, at least where exFAT has been enabled.
- *Long filename overflow in downstream callers*
(CVSS 7.6, High) – With LFN enabled, `fno.fname` can be much larger than many caller buffers expect, as described by CVE-2026-6688. The bug class appears repeatedly in integrations (`strcpy`, `sprintf`, fixed-size name/path fields). This one is tricky to fix entirely in FatFs directly (since exploitation depends on wrappers copying long filenames into undersized local buffers), but it could be mitigated by FatFs changes that make filename lengths and truncation/validation outcomes more explicit to callers.
- *Unsigned-subtraction wrap in dirty-cache handling*
(CVSS 6.1, Medium) – CVE-2026-6685 describes a condition where on fragmented volumes, arithmetic wrap can drive stale dirty-cache behavior and out-of-bounds memory effects in read/write paths. This can manifest as silent corruption, which is exactly the kind of bug operators hate most: hard to detect, easy to misdiagnose, and dangerous in control/data-logging workloads.
- *exFAT divide-by-zero in sync/write paths*
(CVSS 4.6, Medium) – CVE-2026-6683 documents a divide-by-zero condition in exFAT sync/write flows that can be triggered by crafted media and produce reliable crash behavior. In update contexts, this can become a practical bricking vector. While generally more "DoS than RCE," it still offers meaningful attacker value against availability, especially given the OTA implications.
- *Uninitialized cluster exposure after seek beyond EOF*
(CVSS 4.6, Medium) – CVE-2026-6686 describes an uninitialized-cluster exposure path where extending files beyond EOF can leak stale data from previously deleted content. It is an information-disclosure bug with real consequences in multi-stage boot/update and shared-media environments.
- *GPT partition-scan loop DoS (pre-R0.16)*
(CVSS 4.6, Medium) – CVE-2026-6684 covers a GPT entry-count abuse case that can trigger effectively unbounded scanning in older trees. That creates a severe mount-time DoS in affected pre-R0.16 implementations, especially painful in boot paths, but it is ranked last here because upstream R0.16 added protective GPT validation. Now, it's up to implementers to upgrade.

If you have seen XKCD 2347 ("Dependency"), you already know where this is going: one component, maintained in one tiny corner of the internet, quietly supports an absurd amount of modern cyber-stuff.



FatFs is one of those components. It's compact, useful, and copied everywhere. That's great for shipping products quickly, but less great when memory-safety issues show up in parser-adjacent code that happily ingests untrusted media. This kind of component is even more challenging to deal with, from a disclosure-and-fix perspective, in that nearly everyone ends up making local, vendored modifications. So, an upstream patch must be validated pretty carefully before incorporating.

We made repeated attempts to contact the maintainer, and we involved JPCERT/CC early on as well. We never received a response. So, this publication is aimed at the people who can still do something useful right now: downstream implementors. Audit your vendored version, audit your wrappers, audit your file-name and file-size handling, and plan for patch updates.

And now we learn where I obtained the title of today's podcast. HD Moore writes:

*Keeping this class of issue quiet in 2026 would be almost entirely security theater, now that we've entered **the age of the apex agentic adversary**. If we can find this kind of thing with some thoughtful application of AI-assisted vuln-hunting, then so can pretty much anyone else.*

Sure, this took a little HD Moore-branded stubbornness, but even so, we don't believe these are going to stay undiscovered for very long. It's too good a target space, and the next researcher probably won't bother with creating validation and reproduction harnesses.

When it comes to vulnerability disclosure, I've always believed that I'm merely the most recent person to learn about the issue, and that's more true now than ever in this hyper-automated code auditing world. Better to disclose, coordinate where possible, and in the end, publish in order to give defenders the head's up.

PS. Looking for exploits? You can find a test harness, a full qemu-based exploit example, and corrupt FatFs images in the repository.

So, yes... we have indeed entered a brave new world where the rules of the game are clearly changing. The collapsing cost of novel vulnerability discovery means that many new players – both well meaning and malicious – will be joining the fray. The asymmetry of the security guarantee bites us in this instance: Security must get everything right every-where every-time. For insecurity to win, security need only make one mistake in one place once. It's an impossible scenario, but it's the scenario we have today. Someday, thanks to AI, it's clear to me that software will have finally been fixed. And that's something that until now we've only been able to dream of. But to get there from here, we have a long and daunting road ahead. I'm thankful that the world does have many good guy researchers such as HD Moore who are interested and intrigued enough to contribute their time, energy and effort.

In this specific instance I agree with the route Harold took. By the way, the "HD" of "HD Moore" stands for "Harold Dwight." Given that this library is effectively unmaintained, that there's no mailing list nor update methodology and that they weren't even able to contact its author, direct outreach to all of the library's major known users is all the researchers can do. The problem as they note, is that most of the existing installed base is not only currently vulnerable to attack, but almost certainly will never not be.

